

Redis Security Standard

Two secure single-server models: local loopback and TLS-only restricted network

LOCAL SINGLE-SERVER	RESTRICTED NETWORK
Literal loopback profile, ACL identities, no off-host transport.	Private DNS, verified TLS, exact client sources, layered firewall evidence.

PUBLIC SCOPE

Reusable across clients and environments. Deployment names, CA identity, DNS suffixes, addresses, secrets, and evidence remain private inputs.

Version 1.0

August 28, 2026

Redis Open Source 8.2.9 extended release



Contents

Security baseline	3
Reproduce the PDF handoff	3
Application ACL contract	4
Build an immutable deployment bundle	4
Standard 1: local single-server	4
Standard 2: restricted network	5
Add another application identity	6
Verification and evidence	6
Backup, restore, renewal, and change control	7
References	7

This component defines two reusable Redis Open Source deployment standards for Rocky Linux 9 x86-64:

- 1 **Local single-server** - Redis and its only application consumer run on the same host. TCP is bound to 127.0.0.1; a root/redis Unix socket is also available for host administration.
- 2 **Restricted network** - Redis remains a single server, but approved application hosts may connect to one RFC 1918 address. Plaintext TCP is disabled, TLS is mandatory, the certificate identity is a stable private DNS name, and both systemd and external firewalls allow only exact application source addresses.

The standard is client-neutral. Hostnames, address space, CA identities, application names, secrets, and evidence belong in each deployment's private inventory or assurance record, never in this public repository.

Security baseline

Both models enforce the same controls:

- Redis Open Source 8.2.9-1.x86_64, from the official Redis Rocky Linux 9 repository, with the full OpenPGP fingerprint and exact RPM SHA-256 digest pinned before installation.
- Redis runs under the vendor redis account, systemd hardening, and enforcing SELinux. No Linux capabilities or loadable modules are approved. Redis's built-in vectorset implementation may identify itself as a module, but its commands are explicitly denied to application ACLs.
- The default Redis user is disabled. An administrative ACL user has +@all; every application gets a distinct ACL user, secret, key prefix, and Pub/Sub channel prefix.
- Application ACLs allow ordinary reads, writes, transactions, Lua operations, and namespaced Pub/Sub. They deny administrative, dangerous, whole-database, enumeration, and cross-prefix operations. ACL key patterns alone are not treated as isolation for keyless commands.
- Exactly one logical database is enabled. Redis database numbers are not a tenant boundary; isolation is by separate instance plus ACL identity and prefixes.
- maxmemory is explicit and workload-sized, with noeviction. Memory pressure therefore fails writes instead of silently discarding data.
- AOF with appendfsync everysec and RDB snapshots are enabled. This limits a common crash-loss window but does not replace tested backups.
- vm.overcommit_memory=1 and disabled Transparent Huge Pages are acceptance requirements.
- Debug and module commands, runtime protected-configuration changes, keyspace notifications, and unbounded slow-log payload retention are disabled.

Redis is an in-memory service and a single-server deployment has an intentional availability boundary: host or process failure interrupts the service. Use a reviewed high-availability design when the recovery objective cannot tolerate that interruption; do not quietly repurpose this standard as a cluster design.

Reproduce the PDF handoff

This Markdown file is canonical. With ReportLab installed, validate and render the public handoff deterministically from the repository root:

```
redis/build-security-standard-pdf.py --check
redis/build-security-standard-pdf.py
```

The stable output is output/pdf/redis-security-standard.pdf. PDF acceptance also requires rendering every page to images and visually checking typography, wrapping, headers, footers, page references, and placeholder absence.

Application ACL contract

For an application named `example_app` with key prefix `example_app` and channel prefix `example_app`, the managed ACL is equivalent to:

```
user example_app reset on #<sha256-password> resetkeys ~example_app:* resetchannels
&example_app:* -@all +@read +@write +@transaction -@admin -@dangerous -keys -scan -randomkey
-dbsize -vadd -vcard -vdim -vemb -vgetattr -vinfo -vlinks -vrandmember -vrem -vsetattr -vsim
+eval +eval_ro +evalsha +evalsha_ro +publish +subscribe +unsubscribe +psubscribe +punsubscribe
+ssubscribe +sunsubscribe +spublish +ping +echo +hello +quit +client|id +client|getname
+client|setname +client|setinfo
```

The hash is generated locally from a one-line, root-owned mode `0600` password file containing at least 32 characters. Plaintext secrets are never written to Redis configuration, ACL files, plans, logs, or documentation. The password file remains a deployment secret and should be removed from ordinary staging after it is placed in the approved secret manager.

The standard does not promise application compatibility with every Redis command. Additions to the allow-list require a reviewed least-privilege change, an explicit threat analysis for keyless or whole-database behavior, and focused tests.

Build an immutable deployment bundle

Privileged entry points reject a normal developer checkout. Build from a clean, reviewed Git revision:

```
redis/build-bundle --output /absolute/path/config-redis-bundle
```

Copy that directory to a root-owned staging location on the target. Every directory must be owned by `root:root` and not group/other writable; every file must have one hard link, be non-symbolic, and match `SHA256SUMS`.

On the target:

```
sudo /root/config-redis-bundle/redis/install --check-bundle
sudo /root/config-redis-bundle/redis/install
```

The installer leaves `redis.service` stopped and disabled. It never initializes or exposes Redis.

Standard 1: local single-server

Choose dedicated administrative and application identities, generate strong random secrets, and size `maxmemory` below the host's safe Redis allocation. The remaining memory must cover Redis overhead, fork copy-on-write peaks, buffers, the operating system, and every co-resident service.

Review without reading either secret:

```
/root/config-redis-bundle/redis/setup --plan \
--phase initialize \
--model local \
--admin-user redis_admin \
--admin-password-file /root/redis-admin.password \
--application-user example_app \
--application-password-file /root/example-app.password \
--application-key-prefix example_app \
--application-channel-prefix example_app \
--maxmemory-mib 1024
```

Apply only to a new, empty data directory:

```
sudo /root/config-redis-bundle/redis/setup \
--phase initialize \
--model local \
--admin-user redis_admin \
--admin-password-file /root/redis-admin.password \
```

```
--application-user example_app \
--application-password-file /root/example-app.password \
--application-key-prefix example_app \
--application-channel-prefix example_app \
--maxmemory-mib 1024
```

The application connection profile is:

```
scheme: redis
host: 127.0.0.1
port: 6379
database: 0
username: example_app
password: secret-manager reference
TLS: disabled because transport never leaves loopback
```

If an application accepts only a URI, use the equivalent

`redis://example_app:<percent-encoded-password>@127.0.0.1:6379/0`. Treat the entire URI as a secret. Use the literal loopback address, not a hostname whose resolution can drift. Never log a raw URI or copy it into source control.

Standard 2: restricted network

Start with an accepted local model. The transition adds network reachability; it does not weaken ACLs or durability controls.

Before the transition, require all of the following:

- one stable private DNS name reserved for the Redis endpoint;
- DNS A resolution to the exact RFC 1918 bind address from the Redis host and every application host;
- a server certificate whose SAN contains that DNS name, issued by the deployment's approved private or public CA;
- a readable certificate, private key, and complete CA chain at stable absolute paths, with renewal automation and an audited service reload/restart hook;
- the CA trust artifact installed explicitly on every application host;
- exact application source addresses in the host and cloud/network firewalls;
- no public Redis rule and no broad private CIDR when exact sources are known;
- an evidence plan that tests one allowed source and one denied source after the transition.

The setup command validates chain, key match, DNS identity, DNS-to-address resolution, certificate lifetime, Redis account readability, and exact private addresses. The private key must be a one-link root:redis mode 0640 file; certificate and CA files may instead be one-link root:root mode 0644. It sets port 0, tls-port 6379, TLS 1.2/1.3, server authentication, and ACL authentication. Client certificates are not required by this standard; mutual TLS is a separate reviewed profile.

Review:

```
/root/config-redis-bundle/redis/setup --plan \
--phase expose-network \
--admin-user redis_admin \
--admin-password-file /root/redis-admin.password \
--application-user example_app \
--application-password-file /root/example-app.password \
--application-key-prefix example_app \
--application-channel-prefix example_app \
--maxmemory-mib 1024 \
--bind-address 10.20.30.40 \
--server-host redis.internal.example \
--tls-certificate-file /etc/pki/redis/server.crt \
--tls-private-key-file /etc/pki/redis/server.key \
--tls-ca-file /etc/pki/redis/ca-chain.crt \
```

```
--allowed-client-address 10.20.30.21 \  
--allowed-client-address 10.20.30.22 \  
--network-controls-confirmed
```

Apply with the same options and `sudo`. The confirmation means the operator has reviewed the external controls; it is not proof that the command configured them. Setup reports the deployment as pending until allowed-source and denied-source path evidence is recorded.

The application connection profile becomes:

```
scheme: rediss  
host: redis.internal.example  
port: 6379  
database: 0  
username: example_app  
password: secret-manager reference  
CA file: deployment-managed absolute path  
certificate verification: required  
TLS server name: redis.internal.example
```

The equivalent secret URI is

`rediss://example_app:<percent-encoded-password>@redis.internal.example:6379/0`. The CA path and strict hostname verification must be configured explicitly in the client library. Do not use global "accept invalid certificate," disabled hostname validation, IP-literal TLS names, or a system-wide insecure fallback.

Add another application identity

Applications must not share Redis credentials or prefixes. Add a local-model identity with:

```
sudo /root/config-redis-bundle/redis/setup \  
--phase add-application-user \  
--model local \  
--admin-user redis_admin \  
--admin-password-file /root/redis-admin.password \  
--application-user second_app \  
--application-password-file /root/second-app.password \  
--application-key-prefix second_app \  
--application-channel-prefix second_app
```

For the network model, select `--model network` and also provide `--server-host` and `--tls-ca-file`. The command refuses an existing identity, loads the ACL file, and verifies allowed and denied operations.

Verification and evidence

Run the installed verifier with the same identity and prefix values. For an exact configuration comparison, include `--maxmemory-mib`; for the network model also supply the bind, DNS, certificate/key/CA, and a root-controlled file containing one allowed client IPv4 address per line.

The verifier checks:

- exact package version, service state, ownership/modes, SELinux enforcement, systemd policy, and optional exact rendered configuration;
- disabled default user and exact password hashes/ACL rules for the selected administrative and application identities;
- successful application read/write, Lua, and Pub/Sub operations within the namespace;
- denial of unauthenticated access, cross-prefix writes, CONFIG, and KEYS;
- no unapproved loadable modules, one logical database, noeviction, AOF everysec, host memory tuning, and listener scope;

- TLS identity and rejection of plaintext TCP for the network model.

Capture its output, `systemctl status redis`, certificate metadata without private keys, DNS results, firewall rule identifiers, and the dated allowed and denied connection tests in the private deployment evidence set.

Backup, restore, renewal, and change control

- Back up Redis persistence files from a consistent snapshot or a reviewed Redis-aware procedure. Encrypt backups, restrict access, define retention, and keep at least one copy outside the Redis host's failure domain.
- A backup is not accepted until a restore is tested into an isolated instance, application authentication is proven, representative keys are checked, and the recovery time is recorded.
- Monitor AOF/RDB status, failed persistence, rejected connections, memory, fragmentation, latency, certificate expiry, service restarts, and backup success without logging commands, keys, values, passwords, or connection URIs.
- Rotate application passwords one identity at a time. Redis ACLs can hold more than one password during a reviewed overlap, but this component intentionally emits one hash; use a reviewed migration procedure rather than hand-editing the managed ACL.
- CA renewal automation must preserve paths, ownership, modes, SELinux access, and the DNS SAN, then restart Redis and rerun TLS verification. Do not assume a replaced certificate is active merely because a file changed.
- Upgrades require a new reviewed package manifest, full key and digest validation, release-note/security review, backup plus restore proof, staging validation, and a rollback window. Never convert the disabled pinned repository into an unattended update channel.

References

- [Redis security](#)
- [Redis TLS](#)
- [Redis ACLs](#)
- [Redis persistence](#)
- [Redis administration](#)
- [Redis eviction](#)
- [Redis logical databases](#)

Acceptance reminder: restricted-network deployment is incomplete until both an allowed-source test and a denied-source test are recorded in the private evidence set.